

Lecture 17:

Introduction to Deep Learning

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- Recall: Loss Functions, Activation Functions, Gradient Descent, Back Propagation
- Bias-Variance in NNs

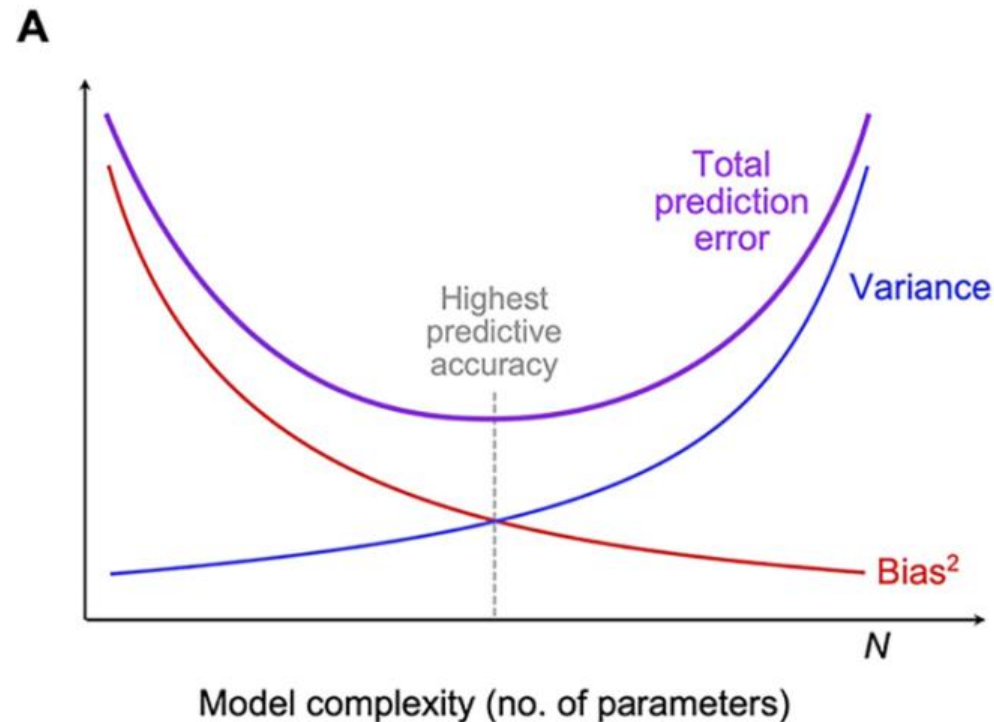
Bias-Variance Tradeoff in Deep Learning

- Why Do Models Fail to Generalize?
- Our goal: learn $f(x) \approx y$ that performs well *on unseen data*
- Training error $\neq$ test error: overfitting/underfitting are two extremes

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Recall: Bias-Variance

- **Bias:** Error due to wrong assumptions (model too simple)
- **Variance:** Error due to too much sensitivity to training data

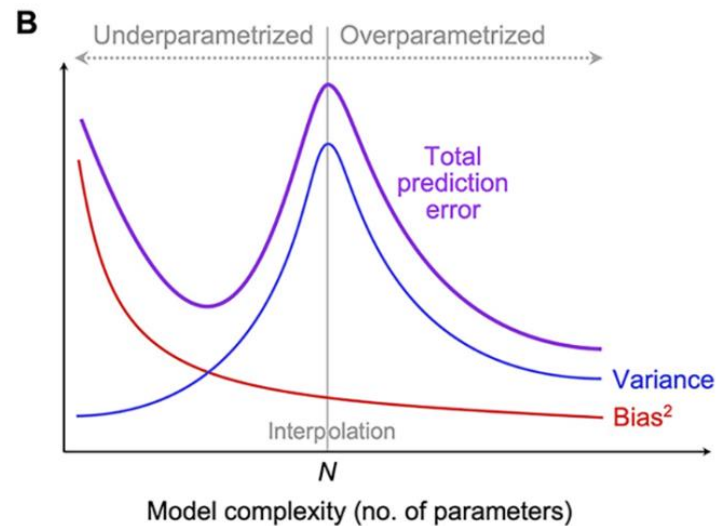


Bias-Variance in Neural Networks

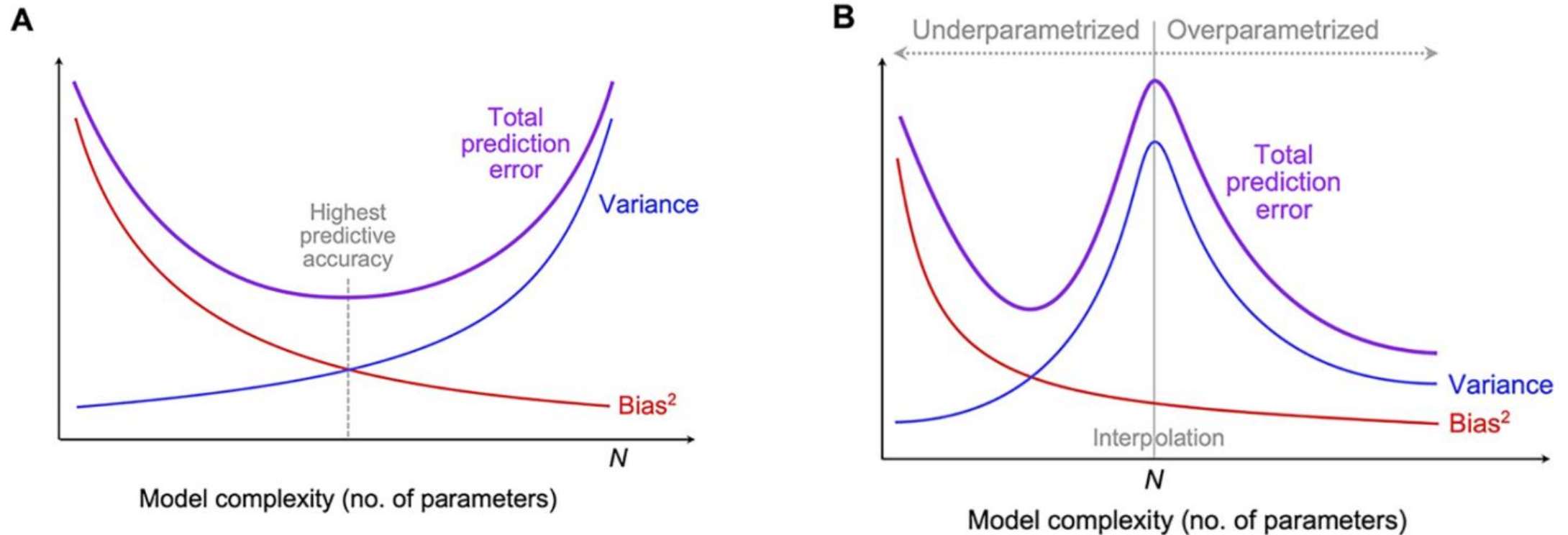
- Shallow networks -----higher bias (can not capture complexity)
- Deep/wide networks ----- lower bias, higher variance
- However, modern deep networks often generalize better than expected (the “double descent” effect)

Beyond Classical Tradeoff - Double Descent Phenomenon

- As model capacity increases:
 - Underparameterized regime --- classical U-shape
 - Interpolation threshold ---- test error peaks
 - Overparameterized regime --- test error drops again
- Seen empirically in deep nets, CNNs, Transformers



Beyond Classical Tradeoff - Double Descent Phenomenon



Three Regions

1. Underparameterized (Low Capacity) Regime

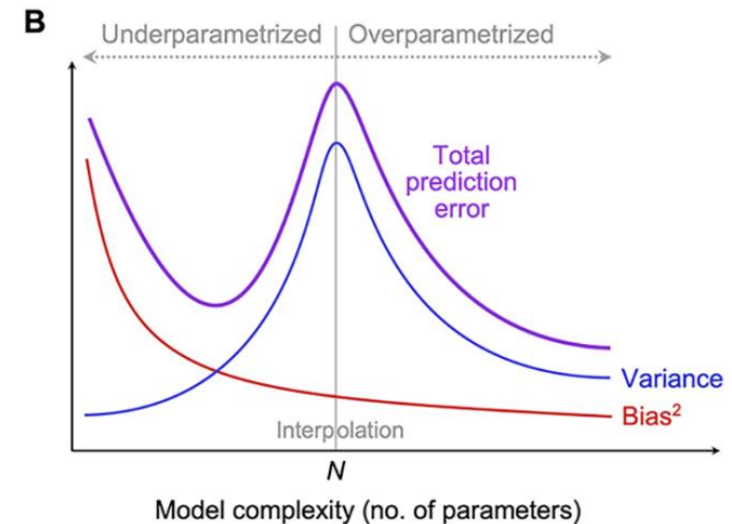
- Model is too simple ---- can not capture data patterns.
- High bias, low variance.
- Training error is high, test error is high.

2. Interpolation Threshold (Critical Capacity)

- Model just large enough to fit all training points perfectly.
- Training error reaches to 0.
- But model “memorizes” noise ---- test error spikes.
- High variance (small data change ---- large prediction change).

3. Overparameterized (High Capacity) Regime

- Model has *way more* parameters than data.
- Many parameter configurations can achieve zero training error.
- Optimization (e.g., SGD) *implicitly biases* the model toward “simpler” or “flatter” solutions that generalize.
- Hence test error *drops again* -- the **second descent**.



How Regularization Balances the Tradeoff

Regularization adds *bias* intentionally to reduce *variance*.

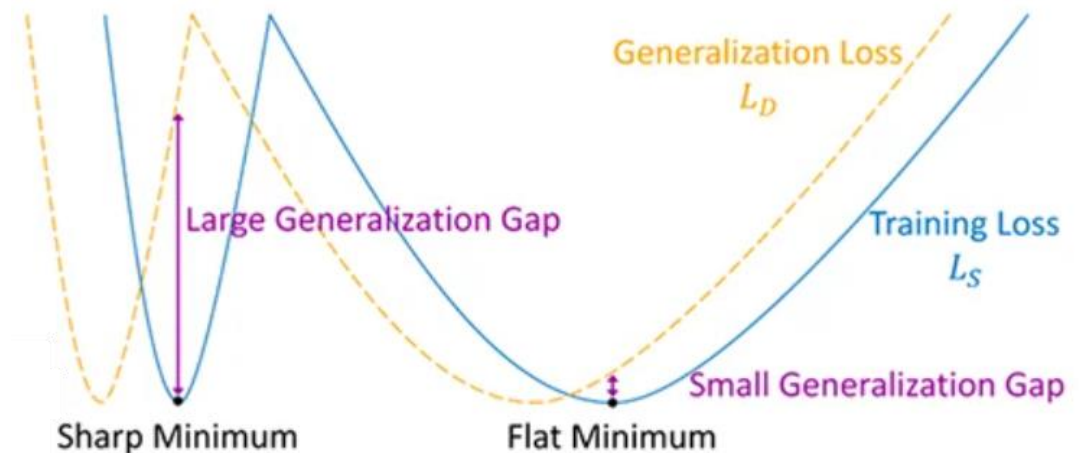
Examples:

- **L2 (Weight Decay):** penalizes large weights --- smoother function
- **Dropout:** reduces co-adaptation between neurons
- **Early stopping:** halts before overfitting begins
- **Data augmentation:** increases effective data diversity

$$\text{Loss} = \frac{1}{N} \sum_i L(y_i, f(x_i; \theta)) + \lambda \|\theta\|_2^2$$

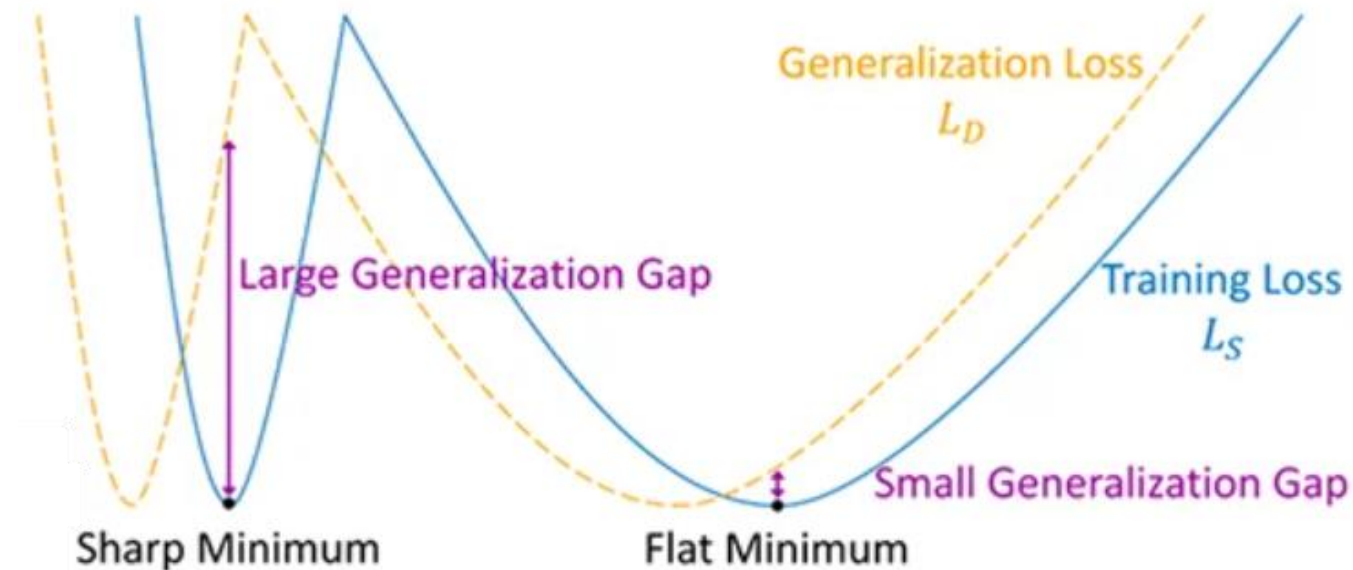
Bias-Variance Behavior in Practice

- Deep nets trained with SGD + noise + dropout often end up in “flat minima” ---- low variance
- Sharp minima ---- high variance (small parameter changes cause large output change)
- Empirically, models with similar training loss can generalize very differently.



Bias-Variance Behavior in Practice

- **Training loss:** measures how well the model fits the data it has seen.
- **Generalization loss:** measures how well the model performs on unseen (test or validation) data drawn from the same underlying distribution



Bias-Variance Behavior in Practice

- Training loss vs. Generalization loss

$$\text{Training Loss: } L_{\text{train}}(\theta) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \ell(f(x_i; \theta), y_i)$$

$$\text{Generalization Loss: } L_{\text{gen}}(\theta) = \mathbb{E}_{(x,y) \sim p_{\text{data}}} [\ell(f(x; \theta), y)]$$

- L_{train} : average loss on the training dataset (finite sample).
- L_{gen} : expected loss over the *true data distribution* p_{data} , what we actually care about.
- Since we can not observe all possible data, we approximate L_{gen} using a **validation or test set**.

How to Manage Bias-Variance Tradeoff?

In deep learning, we want to:

- Reduce **variance** (overfitting) without increasing **bias** (underfitting) too much.
- Achieve a model that generalizes well to unseen data.

1. L1 and L2 Regularization (Weight Penalties)

- **L2 Regularization (Weight Decay)**
- Add a penalty proportional to the squared magnitude of the weights:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i w_i^2$$

- Encourages **small weights** --- smoother functions.
 - Prevents over-reliance on any single feature (reduces variance).
 - Equivalent to adding a Gaussian prior on weights (Bayesian view).
- **In PyTorch:**

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, weight_decay=1e-4)
```

1. L1 and L2 Regularization (Weight Penalties)

- L1 Regularization (Lasso)

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_i |w_i|$$

- Promotes **sparsity** --- drives many weights to exactly zero.
- Leads to simpler, more interpretable models.
- Acts as *feature selection* in linear and shallow networks.

2. Dropout (Stochastic Regularization)

Randomly “drop” neurons during training with a probability p :

$$h^{(l)} = r^{(l)} \odot f(W^{(l)}h^{(l-1)} + b^{(l)}), \quad r^{(l)} \sim \text{Bernoulli}(1 - p)$$

- Each training iteration uses a *different subnetwork*.
- Forces redundancy and discourages co-adaptation between neurons.
- Pytorch example:
- Reduces overfitting (variance decreases).
- Acts similar to an **ensemble** of many smaller networks.
- During inference, weights are scaled by $1 - p$ (no randomness).

```
nn.Dropout(p=0.5)
```


2. Dropout (Stochastic Regularization)

- **Bias-Variance Impact:**
- Adds stochastic noise ----- increases bias slightly.
- Greatly reduces variance ----- improves generalization.

3. Batch Normalization (BN)

- Training deep networks suffers when internal activations shift too much (internal covariate shift).
- For each mini-batch, normalize activations to have zero mean and unit variance:
- where μ_B, σ_B^2 are the batch mean and variance.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$
- Stabilizes learning and allows **higher learning rates**.
- Introduces **slight noise** (different batches --- different normalization), and acts as implicit regularization.
- Makes loss landscape smoother (flatter minima --- better generalization).

- **In PyTorch:**

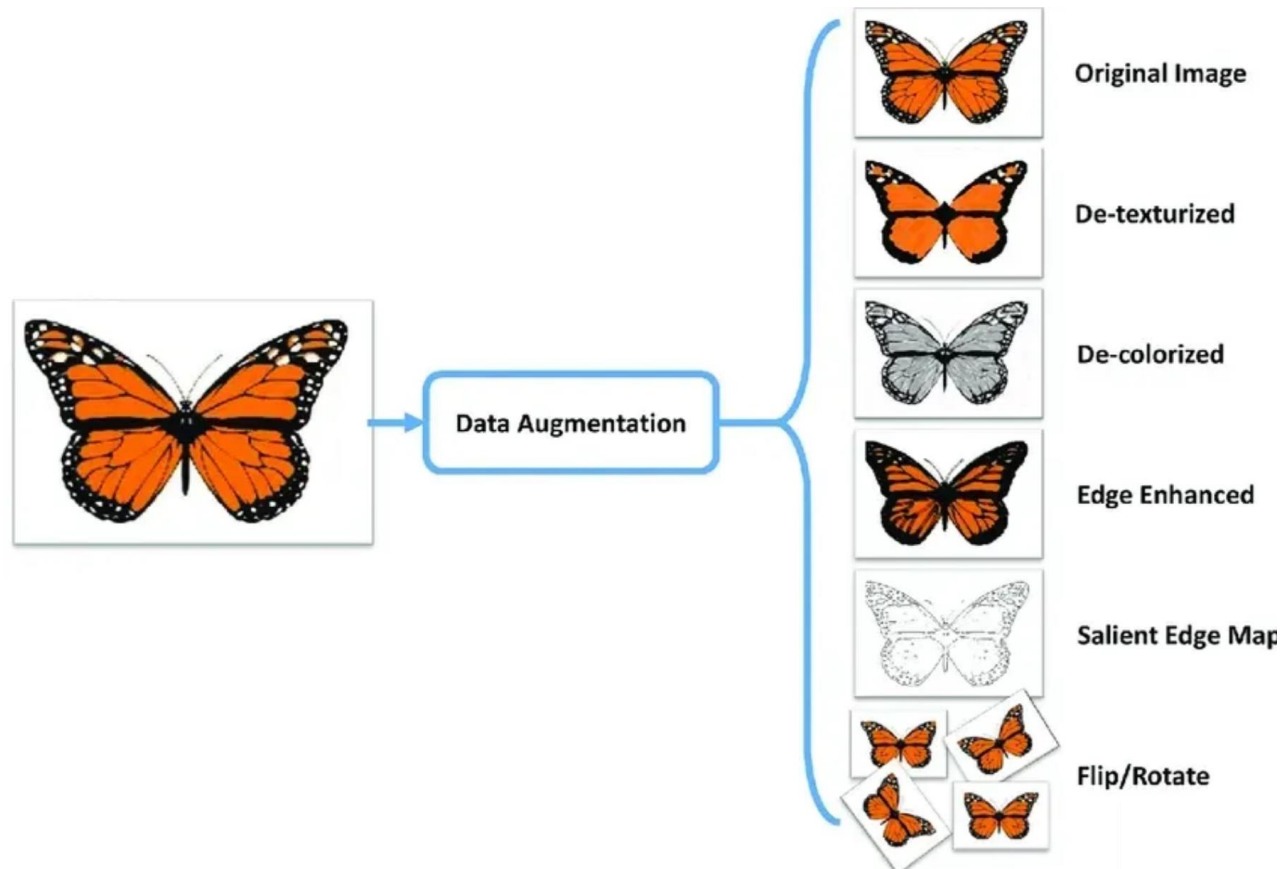
```
nn.BatchNorm1d(num_features)
nn.BatchNorm2d(num_channels)
```

4. Early Stopping

- Stop training when validation loss stops improving.
- Prevents the model from overfitting to training data.
- **Bias-Variance Effect:**
 - Increases bias slightly (stops before full convergence).
 - Reduces variance (prevents memorization).
- **Implementation Tip:** Monitor validation loss: if it has not decreased for N epochs, stop.

5. Data Augmentation

- Artificially expand dataset by applying transformations that preserve labels (e.g., rotations, flips, color jittering for images).



5. Data Augmentation

- **Variance Effect:** When the model sees diverse variations of each sample, it becomes less sensitive to small changes in the data. It learns stable features rather than memorizing pixel-level details.
 - Lower variance, better generalization.
- **Bias Effect:** Slight bias increase (because augmented data slightly changes true input distribution).
- In Pytorch:

```
transforms.RandomHorizontalFlip(),  
transforms.RandomRotation(10)
```

6. Noise Injection (Label or Input Noise)

- Add Gaussian noise to inputs or intermediate activations:

$$x' = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- Forces model to learn smooth mappings.
- Similar in spirit to dropout.

Discussion

- 1) How does dropout influence bias and variance?
- 2) Why can a model with 100M parameters generalize better than one with 1M?
- 3) What happens if you remove regularization entirely in a deep net?